# Fundamentos de Programación

## Grado en Ingeniería de Computadores

18/11/25

# Antecedentes

- Parámetro por defecto
- Paso de función como parámetro
- Funciones lambda
- Ordenaciones alternativas

# Índice

- Construcción de diccionarios
- Agrupación y transformación (Map & Reduce)

# CONSTRUCCIÓN DE DICCIONARIOS

```python
listado = [1, 2, 3, 3, 3, 4, 5, 6, 7, 1]


def construye_diccionario(lista):
    result = dict()

    for dato in lista:
        clave = dato
        if clave in result:
            valor_actualizado = result[dato]+1
            result[clave] = valor_actualizado
        else:
            valor_inicial = 1
            result[clave] = valor_inicial

    return result


print(construye_diccionario(listado))
#{1: 2, 2: 1, 3: 3, 4: 1, 5: 1, 6: 1, 7: 1}
```

# AGRUPACIÓN Y TRANSFORMACIÓN (MAP & REDUCE)



¿Coche más antiguo?



¿Color más frecuente?



```
def hora_mas_avistamientos(avistamientos):
    '''
    Devuelve la hora del día (de 0 a 23) con mayor número de avistamientos

    ENTRADA:
        - avistamientos: lista de tuplas con la información de los avistamientos
            -> [Avistamiento(datetime, str, str, str, int, str, float, float)]
    SALIDA:
        - hora del día en la que se producen más avistamientos -> int

    En primer lugar construiremos un diccionario cuyas claves sean las horas del
    día en las que se han observado avistamientos, y cuyos valores sean el número
    de avistamientos observados en esa hora.
    Después obtendremos el máximo de los elementos del diccionario según el valor
    del elemento.
    '''
```

```python
def agrupa_avistamientos_por_horas(avistamientos: list[Avistamiento])->dict[int, list[Avistamiento]]:
    '''
    Devuelve un diccionario cuyas claves son las horas del día en las que
    se han observado avistamientos, y cuyos valores son las listas de
    avistamientos observados en cada hora.
    :param avistamientos: lista de tuplas con la información de
    los avistamientos
    :return: diccionario con los avistamientos agrupados por hora
    '''
    result = {}
    for a in avistamientos:
        hora = a.fechahora.hour
        if hora not in result:
            result[hora] = []
        result[hora].append(a)
    return result


def transforma_a_frecuencia(avistamientos_por_horas: dict[int, list[Avistamiento]])->dict[int, int]:
    '''
    Devuelve un diccionario con las mismas claves que el recibido, y cuyos
    valores son el número de avistamientos de cada lista.
    :param avistamientos_por_horas: diccionario con los avistamientos
    agrupados por hora
    :return: diccionario con el número de avistamientos de cada hora
    '''
    result = {}
    for hora, lista in avistamientos_por_horas.items():
        result[hora] = len(lista)
    return result


def hora_mas_avistamientos(avistamientos: list[Avistamiento])->int:
    avistamientos_por_horas = agrupa_avistamientos_por_horas(avistamientos)
    result = transforma_a_frecuencia(avistamientos_por_horas)
    return max(result.items(), key = lambda a: a[1])
```

# TAREA

- Completar Avistamientos.
- Completar Aeropuerto Parte 4.